

INTERVIEW — Portfolio Hub (jakeV2)

이 프로젝트(정적 포트폴리오 허브)를 기반으로 나올 수 있는 면접 질문과 모범 답안. 기술, 설계, 트러블슈팅, 압박 질문, 꼬리 질문 까지 대비한다. **답변은 실제 코드 근거로만 구성했다.** 백엔드(NestJS)가 주력인 지원자가, 프론트 전용 프로젝트를 방어하는 상황을 가정한다.

사용법: 각 질문마다 핵심 한 줄 답 → 풀어서 설명 → 예상 꼬리 질문. 소리 내어 연습하라.

A. 프로젝트 개요 / 설계 의도

Q1. 이 프로젝트가 뭐고, 왜 만들었나요?

핵심 답: 제 포트폴리오들을 한 페이지에서 보여주는 개인 랜딩 허브입니다. 단순 이력서가 아니라, 앞으로 프로젝트가 완성될 때마다 코드 수정 없이 데이터만 추가하면 카드가 늘어나는 확장형 구조로 설계한 게 핵심입니다.

풀어서: 채용 담당자가 한 스크롤로 제가 어떤 개발자인지, 스킬과 프로젝트가 뭔지 파악하게 하는 게 목적입니다. 저는 포트폴리오를 계속 만들 계획이라, 허브가 매번 손이 가면 안 됐습니다. 그래서 콘텐츠를 `src/data/` 데이터 파일로 빼고 컴포넌트는 그걸 렌더만 하게 분리했습니다. 새 프로젝트가 완성되면 `projects.ts` 배열에 객체 하나만 추가합니다.

꼬리 질문 대비:

- "S Tier라는데 난이도가 낮지 않나요?" → 네, 기술 난이도는 낮습니다. 대신 저는 설계·타입 안전성·접근성·테스트로 완성도를 채웠습니다. 규모가 작을수록 오히려 과설계를 피하는 판단이 드러난다고 봅니다(→ Q9 참고).

Q2. 왜 프론트엔드 전용인가요? 백엔드가 강점이라면서요.

핵심 답: 이 사이트는 서버에서 받아올 동적 데이터가 없어서 백엔드가 필요 없습니다. 없어도 되는 백엔드를 붙이는 건 과설계입니다.

풀어서: 콘텐츠(프로필·스킬·프로젝트)는 전부 빌드 시점에 고정된 정적 데이터입니다. 사용자 로그인도, DB 조회도, 실시간 갱신도 없습니다. 이런 사이트에 API 서버·DB를 붙이면 운영 비용과 장애 지점만 늘어납니다. 워크스페이스 규약상 프론트 SPA는 Vercel 정적 호스팅으로 배포합니다. 제 백엔드 역량은 이 허브에 연결될 개별 프로젝트들(NestJS 기반)에서 보여줄 계획이고, 허브 자체는 그것들을 담는 관문 역할입니다.

꼬리 질문 대비:

- "그럼 이 프로젝트로 백엔드 역량을 어떻게 어필하죠?" → 이 프로젝트 하나로는 안 합니다. 대신 타입 안전한 데이터 모델링, 스키마 강제, 테스트로 계약 검증 같은 백엔드적 사고를 프론트에 적용한 걸 보여주면 됩니다. `types.ts` 가 DTO/엔티티 스키마 역할을 합니다.

B. 핵심 설계 — 데이터 주도 아키텍처 (가장 중요)

Q3. "데이터 주도 확장 아키텍처"가 정확히 무슨 뜻인가요?

핵심 답: "무엇을 보여줄지(데이터)"와 "어떻게 보여줄지(컴포넌트)"를 분리하고, 컴포넌트는 데이터 배열을 `map` 으로 렌더만 하게 한 구조입니다. 그래서 데이터를 추가하면 컴포넌트를 안 고쳐도 화면이 늘어납니다.

풀어서: 세 층으로 나뉩니다.

- `src/types.ts` — Project 인터페이스로 데이터의 모양(스키마)을 정의.
- `src/data/projects.ts` — 그 타입을 따르는 데이터 배열. 콘텐츠의 유일한 출처(single source of truth).
- `Projects.tsx` / `ProjectCard.tsx` — 배열을 `map` 으로 돌며 카드를 그리는 표현 계층. 안에 하드코딩된 프로젝트 정보가 없습니다.

새 프로젝트 추가는 `projects.ts` 배열에 객체 하나 push. `Projects.tsx` 의 `projects.map` 이 자동으로 카드를 하나 더 그립니다. 이게 SPEC의 성공 기준이자 테스트로 검증하는 동작입니다.

꼬리 질문 대비:

- "map으로 그리는 건 React에선 당연한 거 아닌가요?" → map 자체는 기본이죠. 핵심은 "모든 카드 데이터가 컴포넌트 밖 한 곳(배열)에 있고, 컴포넌트엔 특정 프로젝트 정보가 0개" 라는 점, 그리고 그걸 타입으로 강제하고 테스트로 계약을 고정했다는 점입니다. 하드코딩 카드가 하나라도 있으면 이 구조가 깨집니다.

Q4. 그 설계가 실제로 동작한다는 걸 어떻게 보장하죠?

핵심 답: `Projects.test.tsx` 가 그걸 자동 검증합니다. 데이터를 mock으로 갈아끼워서, 컴포넌트 코드는 그대로인데 데이터만 바꿨을 때 빈 배열이면 "준비 중" 안내, 항목이 있으면 카드가 뜨는지 테스트합니다.

풀어서: `vi.doMock('.../data/projects', ...)` 로 데이터 모듈을 주입합니다. 첫 테스트는 빈 배열을 주입해 `EmptyState` 가 뜨고 `article` (카드)이 없음을 단언하고, 둘째 테스트는 샘플 1개를 주입해 카드 제목이 뜨는지 봅니다. 즉 "데이터→렌더" 계약 자체를 테스트한 겁니다. UI 픽셀이 아니라 아키텍처를 검증한 테스트죠.

꼬리 질문 대비:

- "왜 `import` 를 파일 맨 위가 아니라 테스트 함수 안에서 하나요?" → `vi.doMock` 은 이후 `import`부터 적용됩니다. mock을 건 뒤에 `await import('./Projects')` 로 컴포넌트를 불러와야 mock된 데이터가 들어갑니다. 그래서 `vi.resetModules()` 로 모듈 캐시를 비우고 동적 `import`를 씁니다.
- "`getByText` 와 `queryByText` 왜 섞어 쓰나요?" → `getBy` 는 못 찾으면 에러라 존재 단언에, `queryBy` 는 못 찾으면 null이라 부재 단언(`.not.toBeInTheDocument()`)에 씁니다.

Q5. 카드 리스트의 key 로 왜 slug 를 썼나요? 인덱스는 안 되나요?

핵심 답: 인덱스를 key로 쓰면 목록 순서가 바뀌거나 중간에 삭제될 때 React가 항목을 잘못 매칭해 버그가 생깁니다. `slug` 는 프로젝트마다 고유하고 안정적이라 안전합니다.

풀어서: React는 리스트 재렌더 시 key로 이전/이후 항목을 대응시켜 최소한만 다시 그립니다. 인덱스는 순서에 종속돼서, 앞에 항목을 추가하면 모든 key가 밀려 엉뚱한 재사용이 일어납니다. `slug` ("url-shortener")는 콘텐츠에 종속된 고유 식별자라 순서가 바뀌어도 안정적입니다. 백엔드로 치면 PK를 key로 쓴 셈입니다.

C. React / 프론트엔드 기술

Q6. 이 프로젝트에 `useState` 나 `useEffect` 같은 훅이 하나도 없던데요?

핵심 답: 의도적입니다. 이 사이트는 상태(state)가 전혀 없는 순수 정적 페이지라 훅이 필요 없습니다. 필요 없는 훅을 넣는 게 오히려 안티패턴입니다.

풀어서: 훅은 "컴포넌트가 값을 기억하거나 생명주기에 반응"해야 할 때 씁니다. 그런데 이 사이트엔 사용자 입력으로 바뀌는 값도, 서버에서 받아오는 값도 없습니다. 모든 값이 빌드 시점에 고정입니다. 그래서 컴포넌트들이 전부 입력(props/데이터)을 받아 JSX를 반환하는 순수 함수입니다. 이런 순수 컴포넌트는 예측 가능하고 테스트하기 쉽습니다.

꼬리 질문 대비:

- "훅은 알긴 아냐요?" → 네. `useState` 는 값을 기억하고 그 값이 바뀌면 리렌더를 유발하고, `useEffect` 는 렌더 후 부수효과(데이터 fetch, 구독)를 다룹니다. 예를 들어 이 허브에 프로젝트를 API로 불러오게 바꾼다면 `useEffect` (또는 `TanStack Query`)로 fetch하고 `useState` (또는 쿼리 상태)에 담게 됩니다. 지금은 그럴 필요가 없어 안 쓴 겁니다.
- "그럼 나중에 동적으로 바뀌면요?" → 데이터 소스만 배열에서 fetch로 바꾸면 되고, 컴포넌트의 map 구조는 그대로 재사용됩니다. 그게 지금 분리해둔 이점입니다.

Q7. ProjectCard 에서 props를 어떻게 받고 타입은 어떻게 붙였나요?

핵심 답: `function ProjectCard({ project }: { project: Project })` 로, props 객체에서 `project` 를 구조 분해하면서 그 타입을 `Project` 로 명시했습니다.

풀어서: 부모(`Projects.tsx`)가 `<ProjectCard project={project} />` 로 넘기고, 자식은 매개변수에서 바로 꺼내 씁니다. 타입 `Project` 가 붙어 있어서, 카드 안에서 `project.title` 처럼 오타를 내면 컴파일러가 즉시 잡습니다. 상태 라벨도 `Record<Project['status'], string>` 타입으로 만들어서, status 종류가 늘면 라벨을 빠뜨릴 수 없게 강제했습니다.

Q8. 조건부로 "라이브 데모"/"GitHub" 버튼이 나오던데 어떻게 처리했나요?

핵심 답: `liveUrl / repoUrl` 을 optional 필드(`?`)로 두고, `{project.liveUrl && <a>...}` 로 값이 있을 때만 링크를 렌더합니다.

풀어서: 모든 프로젝트가 라이브 데모나 레포를 가진 건 아닙니다(로컬 쇼케이스 프로젝트도 있음). 그래서 타입에서 optional로 두고, JSX에서 `&&` 단축 평가로 조건부 렌더했습니다. 둘 다 없으면 링크 영역(`<div>`) 자체가 안 생깁니다. 데이터의 유무가 UI를 결정하는, 데이터 주도 방식의 일관된 적용입니다.

D. 압박 질문 — "왜 X 대신 Y를 안 썼나"

Q9. 왜 상태관리 라이브러리(Zustand)나 서버상태(TanStack Query)를 안 썼나요?

핵심 답: 관리할 상태가 없기 때문입니다. 클라이언트 상태도, 서버 상태도 없는 정적 페이지에 상태 라이브러리를 넣으면 순수한 과설계입니다.

풀어서: Zustand는 여러 컴포넌트가 공유하는 클라이언트 상태가 있을 때, TanStack Query는 서버 데이터를 캐싱·동기화할 때 가치가 있습니다. 이 사이트엔 둘 다 없습니다. 워크스페이스에도 "ponytail 원칙 — 필요 없는 의존성 만들지 않기"가 있어서, 정직하게 뺐습니다. 도구는 문제가 있을 때 넣는 것이지, 이력서 채우려고 넣는 게 아니라고 생각합니다.

꼬리 질문 대비:

- "그런 그냥 할 줄 몰라서 아닌가요?" → 다른 프로젝트에서는 씁니다. 워크스페이스 표준 프론트 스택에 Zustand·TanStack Query가 있고, 서버 데이터를 다루는 프로젝트에서 적용할 계획입니다. 여기서 뺀 건 **역량 부족**이 아니라 판단입니다. 필요를 만들어서 넣는 것과 필요할 때 넣는 것의 차이입니다.

Q10. 라우터(React Router)는 왜 안 썼나요? 프로젝트 상세 페이지가 있으면 좋지 않나요?

핵심 답: 단일 스크롤 페이지 + 앵커 네비게이션으로 충분하고, 프로젝트 상세는 외부 링크(라이브/GitHub)로 이동시키기 때문입니다.

풀어서: 섹션이 4개뿐이라 `#about`, `#projects` 같은 앵커로 부드럽게 스크롤(`scroll-behavior: smooth`)하는 게 더 단순하고 빠릅니다. 프로젝트 상세는 각자 별도 배포된 라이브 데모/레포로 연결하므로 허브 안에 상세 라우트를 둘 이유가 없습니다. 라우터를 넣으면 코드 스플리팅·404 처리 등 부수 복잡도만 늘어납니다.

꼬리 질문 대비:

- "나중에 상세 페이지가 필요해지면요?" → 그때 `react-router` 를 도입하고 프로젝트별 상세 라우트를 추가하면 됩니다. 지금 데이터가 `slug` 를 갖고 있어서 `/projects/:slug` 라우팅으로 확장하기 쉽게 이미 준비돼 있습니다.

Q11. Tailwind 대신 CSS Modules나 styled-components를 쓰지 않은 이유는?

핵심 답: Tailwind는 별도 CSS 파일 없이 반응형·다크 대비를 빠르게 처리할 수 있고, 워크스페이스 표준이라 프로젝트 간 일관성이 있습니다.

풀어서: styled-components는 런타임 CSS-in-JS라 약간의 런타임 비용이 있고, CSS Modules는 클래스 파일을 따로 관리해야 합니다. Tailwind는 유틸리티 클래스로 JSX 안에서 바로 스타일을 주고, md: lg: 접두사로 반응형을, @theme 토큰으로 디자인 시스템을 한 곳에서 관리합니다. 다만 단점(→ Q16)도 인지하고 있습니다.

Q12. Next.js가 요즘 표준인데 왜 그냥 Vite + React인가요?

핵심 답: 이 사이트엔 SSR·서버 컴포넌트·파일 기반 라우팅·API 라우트가 필요 없어서, 더 가벼운 Vite로 충분합니다.

풀어서: Next.js의 강점은 SSR/SSG, 서버 컴포넌트, 이미지 최적화, API 라우트인데, 이 정적 단일 페이지는 그 기능이 하나도 필요 없습니다. Vite는 개발 서버가 매우 빠르고 설정이 간단해 이 규모에 적합합니다. SEO가 중요하지만 콘텐츠가 초기 HTML에 대부분 담기고 규모가 작아, Next.js의 SSR까지 갈 필요는 없다고 판단했습니다.

꼬리 질문 대비:

- "SPA는 SEO에 불리하지 않나요?" → 순수 CSR SPA는 초기 HTML이 비어 크롤러에 불리할 수 있습니다. 이 사이트는 규모가 작고, 개선이 필요하면 Vite 정적 프리렌더링이나 메타 태그(OG) 보강으로 대응할 수 있습니다. 트래픽·검색 요구가 커지면 그때 Next.js 마이그레이션을 검토하겠습니다.

E. 설계의 단점 / 트레이드오프 (솔직함 테스트)

Q13. 이 설계의 단점은 뭔가요?

핵심 답: 콘텐츠를 바꾸려면 코드를 수정하고 재배포해야 한다는 점입니다. 비개발자가 CMS로 편집할 수 없습니다.

풀어서: 데이터가 .ts 파일에 있어서, 프로젝트를 추가하려면 코드를 커밋하고 Vercel이 다시 빌드·배포해야 합니다. 저(개발자) 혼자 운영하는 개인 사이트라 이걸 오히려 장점(타입 안전, 버전 관리, 무료 정적 호스팅)이지만, 여러 명이 콘텐츠를 자주 바꾸는 상황이라면 헤드리스 CMS나 관리 화면이 필요했을 겁니다. 저는 **운영자가 나 하나**라는 맥락에서 이 트레이드오프를 의도적으로 선택했습니다.

꼬리 질문 대비:

- "그럼 CMS로 바꾸는 게 낫지 않나요?" → 요구가 바뀌면요. 지금은 오버킬입니다. 다만 데이터 계층이 이미 분리돼 있어서, projects.ts 의 배열을 CMS API 호출로 교체하고 컴포넌트는 그대로 두는 식으로 마이그레이션할 수 있습니다. 분리해둔 덕에 확장 경로가 열려 있습니다.

Q14. 다크모드 고정인데, 라이트 모드 사용자는요?

핵심 답: 콘솔/터미널 무드의 디자인 컨셉이라 다크 단일 테마로 고정했고, 색 대비는 WCAG AA를 목표로 맞췄습니다.

풀어서: 디자인 시안(Console 무드)이 다크 기반이고, 테마 토큰은 이 규모에서 과설계라 했습니다(color-scheme: dark 고정). 다만 접근성은 포기하지 않아서, 텍스트/배경 대비를 충분히 확보했습니다. 라이트 모드가 요구되면 @theme 토큰을 CSS 변수로 이미 관리하므로 라이트 팔레트를 추가하고 prefers-color-scheme 로 분기하면 됩니다.

Q15. 프로젝트 배열이 비어 있는데, 이 상태로 면접에 들고 오면 빈 사이트 아닌가요?

핵심 답: 빈 배열일 때 "포트폴리오 프로젝트 준비 중" 빈 상태 UI가 정상 렌더되도록 처리했고, 이 빈 상태 처리 자체가 성공 기준이자 테스트 항목입니다.

풀어서: 빈 상태를 방치해 화면이 깨지는 게 아니라, EmptyState 컴포넌트로 의도된 안내를 보여줍니다. 이걸 실무에서 "데이터 없음/로딩/에러" 상태를 빠뜨리지 않는 습관을 보여주는 지점입니다. 실제 프로젝트들이 완성되면 배열에 채워지고, 그 흐름은 테스트로 이미 검증돼 있습니다.

Q16. Tailwind 유틸리티 클래스가 JSX를 지저분하게 만들지 않나요?

핵심 답: 맞습니다. 클래스 문자열이 길어져 가독성이 떨어지는 건 Tailwind의 알려진 단점입니다. 저는 반복되는 UI를 컴포넌트 (`SectionHead` , `ProjectCard`)로 추출해 이 문제를 완화했습니다.

풀어서: 클래스가 길어지는 걸 막는 정공법은 **반복을 컴포넌트로 뽑는 것**입니다. 섹션 헤더가 여러 번 나오니 `SectionHead` 로, 카드가 여러 개니 `ProjectCard` 로 추출해서 스타일을 한 곳에 모았습니다. 그래서 같은 긴 클래스를 복붙하지 않습니다. 더 커지면 `@apply` 로 자주 쓰는 조합을 클래스로 묶는 방법도 있습니다.

F. 접근성 / 품질

Q17. 접근성을 위해 뭘 했나요?

핵심 답: 시맨틱 랜드마크 + `aria-labelledby` , 스킵 링크, 키보드 포커스 링, `sr-only` 패턴, `prefers-reduced-motion` 존중, 외부 링크 `rel="noopener noreferrer"` 입니다.

풀어서: (하나씩) `<header>/<main>/<section>/<footer>` 로 구조를 의미화하고 각 섹션을 제목과 `aria-labelledby` 로 연결했습니다. 키보드 사용자를 위해 "본문 바로가기" 스킵 링크를 넣어 포커스 시 나타나게 했고, 모든 링크에 앰버 포커스 링(`focus-visible`)을 유지했습니다. Header에서는 모바일에서 라벨을 시각적으로 숨기되(`sr-only sm:not-sr-only`) 스크린리더엔 전체 텍스트가 읽히게 했습니다. 애니메이션은 `prefers-reduced-motion` 으로 끌 수 있고, 새 탭 링크엔 tabnabbing 방지를 위해 `rel` 을 붙였습니다.

꼬리 질문 대비:

- " `sr-only sm:not-sr-only` 가 정확히 뭐죠?" → 작은 화면(< sm)에선 `sr-only` (화면엔 안 보이고 스크린리더만 읽음)로 라벨을 숨겨 네비를 한 줄로 유지하고, sm 이상에선 `not-sr-only` 로 다시 보이게 합니다. 시각적 공간과 접근성을 동시에 만족시키는 기법입니다.
- " `rel="noopener noreferrer"` 가 막는 게 뭔가요?" → `noopener` 는 새 탭이 `window.opener` 로 원본 페이지를 조작하는 tabnabbing 공격을, `noreferrer` 는 referrer 정보 유출을 막습니다.

Q18. 반응형은 어떻게 처리했나요?

핵심 답: 모바일 퍼스트로, Tailwind 브레이크포인트 접두사(`md:` , `lg:`)를 써서 프로젝트 그리드를 1열 → 2열 → 3열로 늘립니다.

풀어서: 기본은 모바일(1열), md (태블릿)에서 `md:grid-cols-2` , lg (데스크톱)에서 `lg:grid-cols-3` . 컨테이너는 `max-w-5xl mx-auto` 로 폭을 제한하고 좌우 패딩을 반응형(`px-5 sm:px-8`)으로 줍니다. Hero 제목은 `clamp()` 로 화면 크기에 따라 폰트가 유동적으로 커지고, 파이프라인 SVG는 고정 폭 대신 `width="100%"` + `viewBox` 로 좁은 화면에서 잘리지 않게 했습니다(실제로 겪은 이슈 → Q20).

G. 빌드 / 배포 / 트러블슈팅

Q19. 빌드·배포 파이프라인을 설명해 주세요.

핵심 답: `tsc -b && vite build` 로 타입검사 통과 후 정적 자산을 `dist/` 에 뽐고, Vercel이 그걸 CDN에 배포합니다.

풀어서: `build` 스크립트가 먼저 `tsc -b` 로 전체 타입을 검사합니다. 타입 에러가 하나라도 있으면 `&&` 뒤가 실행되지 않아 빌드 자체가 실패합니다. 즉 타입 안전성이 배포 게이트입니다. 통과하면 `vite build` 가 코드·CSS·폰트를 번들·압축해 정적 파일로 만들고, Vercel이 정적 호스팅 + CDN으로 서버합니다(<https://jakev2.vercel.app>). 서버가 없으니 콜드스타트나 스케일링 걱정이 없습니다.

Q20. 개발하면서 겪은 문제와 해결은?

핵심 답: Hero의 데이터 파이프라인 SVG가 좁은 화면에서 세 번째 노드가 잘리는 문제가 있었습니다. 고정 `min-width` 대신 `width="100%"` + `viewBox` 로 완전 유동적으로 축소되게 해서 해결했습니다.

풀어서: SVG에 고정 폭을 주니 모바일에서 내부 가로 스크롤이 생기고 마지막 노드(`postgres`)가 가려졌습니다.

`viewBox="0 0 640 90"` 로 좌표계를 고정하고 `width="100%"` , `preserveAspectRatio="xMidYMid meet"` 를 줘서 컨테이너 폭에 맞춰 비율 유지하며 축소되게 했습니다. 또 `body`에 `overflow-x: hidden` 을 걸어 어떤 자식도 가로 넘침을 못 만들게 방어했습니다. 코드 주석에도 이 의도를 남겼습니다.

꼬리 질문 대비:

- "테스트로 잡을 수 있었나요?" → 레이아웃/시각 회귀는 단위 테스트로 잡기 어렵습니다. 실제 뷰포트에서 확인해야 하는 종류라, 여러 폭에서 수동 확인했습니다. 규모가 커지면 Playwright 시각 회귀 테스트를 붙이는 게 정공법입니다.

Q21. 테스트는 어떻게 구성했고 무엇을 검증하나요?

핵심 답: Vitest + React Testing Library로, 데이터 주도 렌더링 계약을 검증합니다. 데이터가 비면 빈 상태, 있으면 카드가 뜨는지.

풀어서: (Q4와 연결) 이 프로젝트에서 가장 중요한 불변식은 "데이터→렌더"입니다. 그래서 픽셀이 아니라 그 계약을 테스트했습니다. RTL은 사용자 관점(텍스트/역할)으로 요소를 찾고, `vi.doMock` 으로 데이터를 주입해 컴포넌트 코드 불변을 전제로 결과가 바뀌는지 봅니다. 백엔드에서 서비스 계약을 통합 테스트하는 것과 같은 사고를 프론트에 적용한 겁니다.

H. 마무리 / 성장 질문

Q22. 이 프로젝트에서 배운 점은?

핵심 답: "작게 만듦이 확장 가능하게" — 과설계를 피하면서도 미래 확장 경로를 데이터 계층 분리로 열어두는 균형 감각을 배웠습니다.

풀어서: 백엔드에서 익숙한 "스키마로 계약을 강제한다"는 사고를 프론트 데이터 모델에 적용하니, 콘텐츠 확장이 안전하고 예측 가능해졌습니다. 또 상태 라이브러리·라우터를 "안 넣는" 판단도 설계라는 걸 체감했습니다. 넣는 것보다 안 넣는 걸 정당화하는 게 더 어렵더군요.

Q23. 지금 다시 만든다면 뭘 바꾸겠어요?

핵심 답: 실제 프로젝트가 쌓이면 경력/프로젝트 상세를 위해 라우팅과 OG 메타·프리렌더링(SEO)을 보강하고, 시각 회귀 테스트(Playwright)를 붙이겠습니다.

풀어서: 지금 구조가 그걸 쉽게 해줍니다. `slug` 기반 상세 라우트, 데이터 소스를 CMS/파일로 교체, 프리렌더로 SEO 개선 — 전부 데이터·표현 분리 덕에 컴포넌트 큰 수정 없이 갈 수 있습니다. 다만 이건 요구가 실제로 생겼을 때 하겠습니다. 지금 미리 넣는 건 또 과설계니까요.

부록: 30초 엘리베이터 피치

"제 포트폴리오를 모아 보여주는 랜딩 허브입니다. 핵심은 **콘텐츠(데이터)와 표현(컴포넌트)을 분리**해서, 새 프로젝트가 완성되면 `projects.ts` 배열에 객체 하나만 추가하면 **컴포넌트 수정 없이 카드가 자동으로** 뜨는 구조라는 점입니다. TypeScript로 데이터 스키마를 강제하고, 그 '데이터→렌더' 계약을 Vitest 테스트로 검증했습니다. 상태 라이브러리나 라우터는 필요가 없어서 의도적으로 뺐고, 접근성(시맨틱 태그·스킵 링크·키보드 포커스)과 반응형은 기본으로 챙겼습니다. Vercel에 정적 배포돼 있습니다."